Assessment cover

Module No:	COMP5047	Module title:	Applied Software Engineering

Assessment title:	Resit Coursework - Software Engineering of a Modern Computer Application

Due date and time:	9:00am, 14th April, 2025

Estimated total time to be spent on assignment:	84 hours per student

LEARNING OUTCOMES

On successful completion of this assignment, students will be able to achieve the module's following learning outcomes (LOs):
1. Demonstrate an understanding of the role of requirements analysis and specification in software engineering and to be able to use this knowledge to create use case models and functional models of computer applications.
2. Demonstrate an understanding of the relationship between requirements and design and to be able to apply the knowledge to create structural and behavioural models of computer applications.
3. Critically evaluate and utilise design paradigms of object-oriented analysis and design, component-based design, and service-oriented design.
4. Use software modelling language such as UML and modelling tools in the context of model-driven software engineering.
5. Work in a group to apply the knowledge and skills developed in this module

Engineering Council AHEP4 LOs assessed	
C3	Select and apply appropriate computational and analytical techniques to model complex problems, recognising the limitations of the techniques employed
C5	Design solutions for complex problems that meet a combination of societal, user, business and customer needs as appropriate. This will involve consideration of applicable health & safety, diversity, inclusion, cultural, societal, environmental and commercial matters, codes of practice and industry standards
C6	Apply an integrated or systems approach to the solution of complex problems
C14	Discuss the role of quality management systems and continuous improvement in the context of complex problems
C16	Function effectively as an individual, and as a member or leader of a team

Student Name: Ali Aizhigitov **Student Id:19271590** **Subsystem: CloudTables-Customer**

Statement of Compliance *(please tick to sign)*

☑ I declare that the work submitted is my own and that the work I submit is fully in accordance with the University regulations regarding assessments (*www.brookes.ac.uk/uniregulations/current*)

RUBRIC OR EQUIVALENT:

Marking grid and marking form are available on Moodle website of the module.

FORMATIVE FEEDBACK OPPORTUNITIES

(a) Discuss your work with your practical class tutor during practical classes; (b) Discuss your work with lecturer and/or practical class tutor in drop-in hours.

SUMMATIVE FEEDBACK DELIVERABLES

Deliverable content and standard description and criteria
Please see attached file of *COMP6030 Coursework Marking and Feedback* for feedbacks on your coursework, which include:
(a) Breakdown of marks on each assessment criterion
(b) Comments on each aspect of the assessment against assessment criteria
(c) Annotations on your submitted work

Task 1: Project Management and SE Tools

1.1. Name of the UML Modelling Tool Used in Each Task

Instead of using a graphical UML tool such as UML Designer, PlantUML was used throughout the rest of the coursework to define UML diagrams in a textual format. This method ensured syntactically accurate and reusable diagrams that were fully aligned with the specification and marking criteria. PlantUML source code files (.puml) and their corresponding image exports (.png) are included in the submission.

1.2. URL of GitHub Repository

https://github.com/AliAizhigitov/cloudtables-customer-resit

1.3. List of Folders and Files on GitHub Repository

COMP5047_ResitCW_19271590.pdf

Task1_ToolsAndManagement.docx

Task2_QualityRequirements.docx

Task3_FunctionalModel.docx

Task4_ArchitectureDesign.docx

Task5_DetailedDesign.docx

UseCaseDiagram.puml

UseCaseDiagram.png

ActivityDiagram.puml

ActivityDiagram.png

ComponentDiagram.puml

ComponentDiagram.png

ClassDiagram.puml

ClassDiagram.png

SequenceDiagram.puml

SequenceDiagram.png

Task 2: Specification of Quality Requirements

Subsystem: CloudTables-Customer

Use Case Focus: Booking a Table (FR-RC-2)

(a) Security and Privacy

1. All communications between the customer-facing frontend and backend services must be secured using HTTPS with TLS 1.3 encryption. This ensures that sensitive user data, including login credentials and booking details, are not exposed during transmission. TLS termination will be handled by the NGINX ingress controller deployed within the Kubernetes cluster, ensuring secure connectivity from the browser to the API Gateway.

2. User authentication is managed via JSON Web Tokens (JWTs). Upon successful login, the server generates a signed JWT containing the user's ID and role. The token has a 15-minute expiration period to minimize security risks. All requests to protected endpoints must include the token in the `Authorization` header. Verification is performed at the API Gateway layer for each request.

3. Access to backend microservices is restricted using Role-Based Access Control (RBAC). JWTs include a `role` claim, which is validated against the endpoint's required access level. For example, only users with the `customer` role are permitted to perform actions such as booking a table or submitting a review.

4. All critical actions related to bookings are logged in an audit trail. This includes actions like table reservation, booking updates, cancellations, and payment attempts. Logs include timestamp, user ID, action type, and request metadata. These logs are forwarded to a centralized logging stack (e.g., ELK or CloudWatch Logs) for security auditing and incident investigation.

5. Personally Identifiable Information (PII) such as customer email addresses and phone numbers is encrypted at rest using AES-256. PostgreSQL is configured with the `pgcrypto` extension to enable field-level encryption. Encryption keys are rotated quarterly and managed securely via Kubernetes secrets or a cloud KMS (Key Management Service).

Testability Measures:
- Conduct OWASP ZAP scans to ensure no high-risk vulnerabilities.
- Use Postman to test access to protected routes with invalid/expired JWTs.
- Verify that all booking actions are properly recorded in the centralized audit log.

(b) Performance

1. Table search operations must return results within 1 second for 95% of user requests. To achieve this, restaurant and table metadata is indexed using PostgreSQL's GIN index structures, and commonly accessed queries are cached using Redis with a TTL of 60 seconds.

2. Booking confirmation workflows must be completed within 2 seconds from the user's request to the delivery of confirmation. This is facilitated through asynchronous task processing using RabbitMQ, which decouples the user-facing request from downstream services like notifications and availability checks.

3. The booking service must support 500 concurrent users without queuing requests or returning 503 errors. This is enforced through Kubernetes Horizontal Pod Autoscaler (HPA), which scales the number of replicas dynamically based on CPU and request latency.

4. Pre-order food items are retrieved using a paginated GraphQL endpoint that serves lightweight responses (e.g., 10 items per page). Each page load is expected to complete in under 1.5 seconds. Images and static assets are delivered via CloudFront CDN.

5. Under normal operating load, all microservices must respond to HTTP requests within 300 milliseconds. Performance is monitored using New Relic and Prometheus metrics dashboards, with alerts triggered when latency exceeds thresholds.

Testability Measures:
- Simulate 500+ concurrent sessions using Apache JMeter or k6.
- Benchmark response times using real-time monitoring tools.
- Validate caching effectiveness using Redis key access logs.

(c) Reliability

1. Booking operations must achieve 99.95% uptime on a monthly basis. This is ensured by deploying microservices on a highly available Kubernetes cluster with node pools spanning multiple availability zones.

2. In case of transient failures, the frontend application includes retry logic using Axios interceptors that will automatically reattempt failed booking requests once after a 5-second delay.

3. The Booking Service is configured with a minimum of 2 replicas at all times. These are registered behind a Kubernetes Service object with built-in round-robin load balancing. If a pod fails, Kubernetes automatically reschedules it to maintain high availability.

4. The system is monitored using Prometheus and Grafana, with alerts configured through Alertmanager and PagerDuty. Critical metrics such as HTTP error rates, memory usage, and pod restarts are monitored in real time.

5. All booking-related and user-generated data (pre-orders, reviews) are backed up nightly using AWS RDS automated snapshots. Snapshots are replicated to a secondary region for disaster recovery and retained for 14 days.

Testability Measures:
- Perform chaos engineering using tools like Chaos Monkey.
- Review alert logs and backup consistency reports.
- Simulate pod crashes to verify rescheduling behavior.

(d) Scalability

1. The Booking Service is stateless and can be horizontally scaled using Kubernetes HPA. Metrics such as CPU usage and request latency are used to auto-scale pods from a baseline of 2 up to 10 replicas during peak load.

2. The API Gateway, built using NGINX or Kong, is deployed as a stateless component and can be replicated independently. It is fronted by a LoadBalancer service and configured to scale separately from other services.

3. PostgreSQL is configured to support read scalability using streaming replication to read-only replicas. Additionally, the `booking` table is partitioned by date to allow parallel read access and minimize query planning time.

4. The React-based frontend uses code-splitting with `React.lazy` and `Suspense` to load only the components needed on each route. Menu data is fetched asynchronously using GraphQL queries that support offset-based pagination.

5. The platform is validated to handle up to 10x the nominal user load (5000 concurrent users). Load tests using Locust simulate real traffic with scenarios that mimic login, search, booking, and pre-order actions. Autoscaling and response metrics are logged and analyzed after each test round.

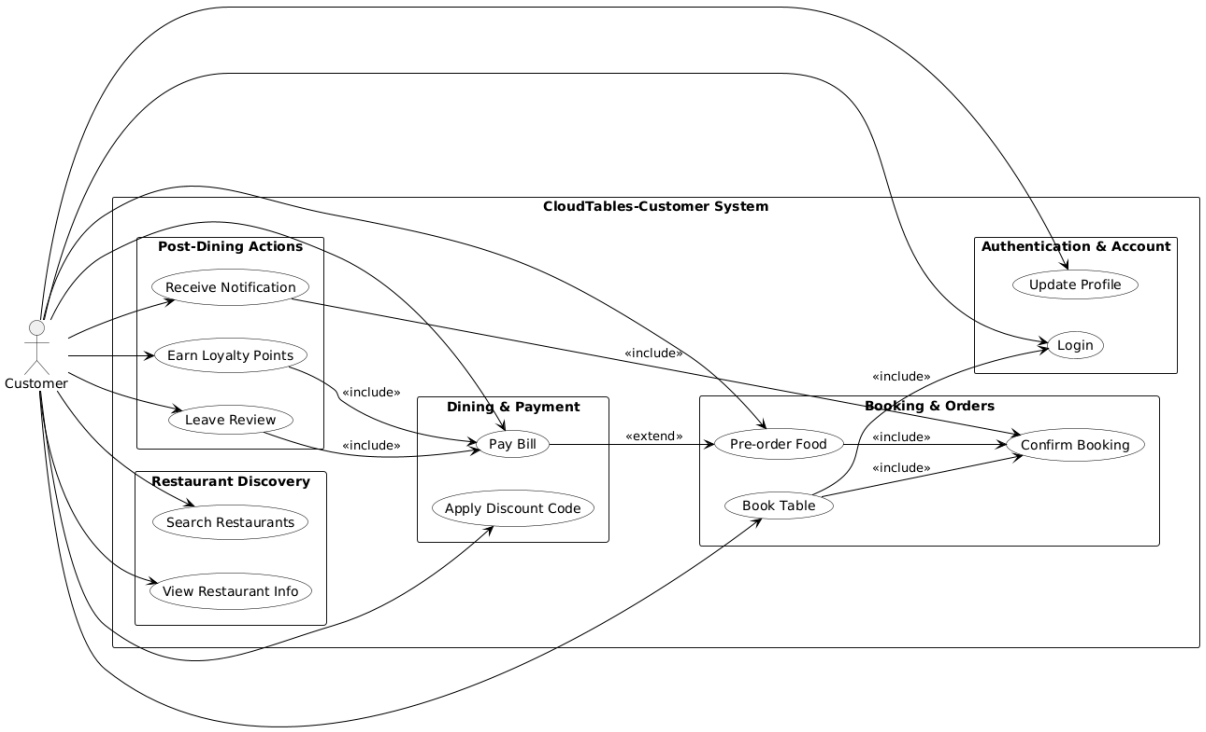
Testability Measures:
- Monitor pod scaling activity during test loads.
- Track query performance and API latency under stress.
- Confirm frontend responsiveness and load times at high concurrency.

Task 3: Functional Model

Use Case Focus: Booking a Table

(a) UML Use Case Diagram and Description

This UML Use Case Diagram models the functional boundaries of the CloudTables-Customer subsystem. It defines how the Customer actor interacts with the system's main features, represented as use cases.



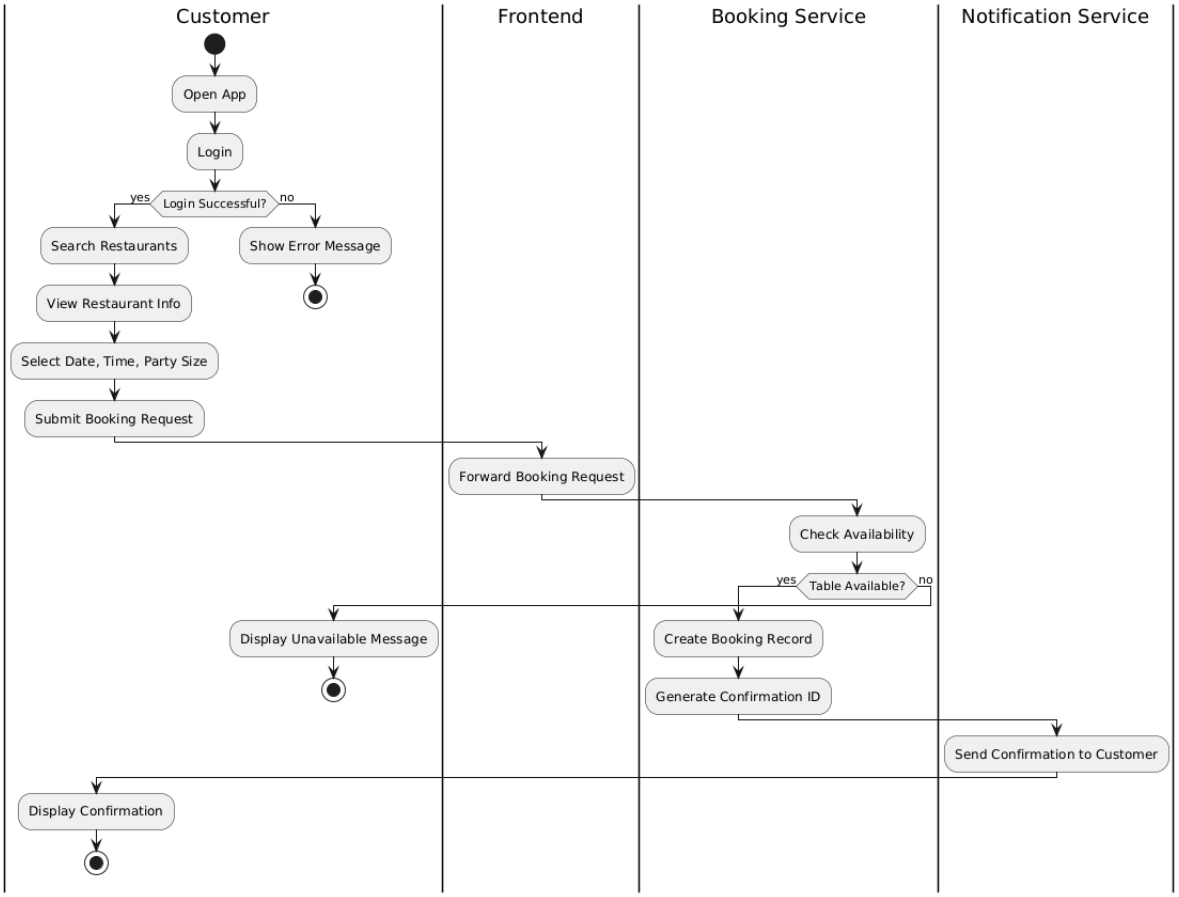
Use Cases Included:
- Login – A required use case that precedes other services.
- Search Restaurants – Allows filtering by location, name, or cuisine.
- View Restaurant Info – Presents menus, reviews, and table availability.
- Book Table – The main use case; includes login and confirmation.
- Confirm Booking – Represents the internal system's finalization of the booking process.
- Pre-order Food – Optional use case that customers may include during booking.
- Pay Bill – Involves digital payment; can optionally follow pre-order.
- Leave Review – Only available once a bill has been paid.

Relationships Modeled:
- <<include>>: Login in Book Table; Confirm Booking in Book Table and Pre-order Food; Pay Bill in Leave Review.
- <<extend>>: Pre-order Food is extended by Pay Bill (optional path).

System Scope: The system boundary is labeled as 'CloudTables-Customer System' and encloses all the use cases offered to the Customer. It defines what is provided by the subsystem and ensures external responsibilities are not mixed in.

(b) UML Activity Diagram and Description

The UML Activity Diagram below details the flow of actions when a customer books a table. It distinguishes between customer actions and system processes using swimlanes and shows the sequence of steps, decisions, and outcomes.



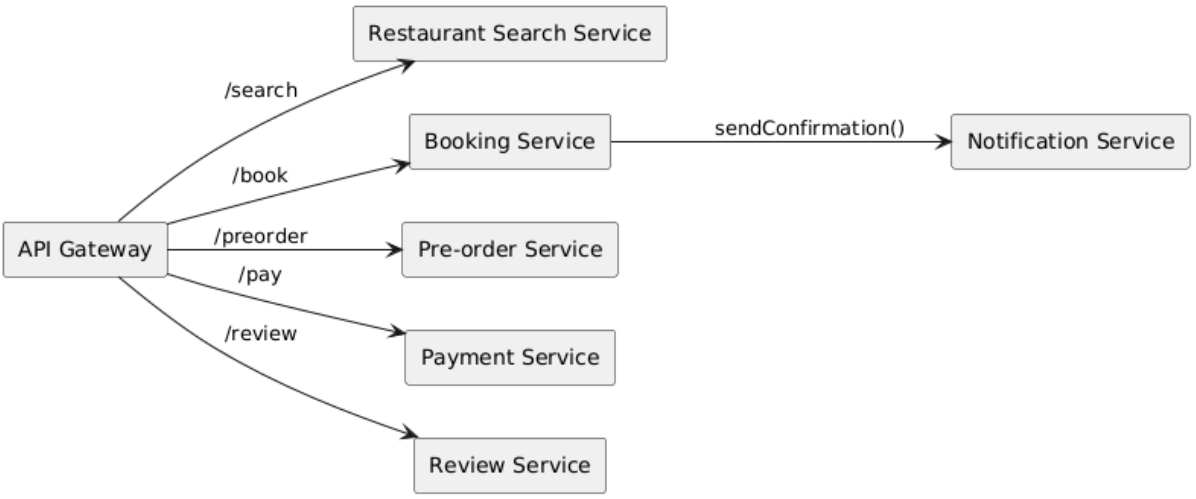
Flow Narrative:
Customer Swimlane:
1. Login – The customer authenticates using credentials.
2. Search Restaurants – Filters restaurant options.
3. View Info – Reads details, menus, and available slots.
4. Select Time & Table – Chooses date, time, and number of guests.

System Swimlane:
5. Check Availability – The system queries table availability for the selected slot.
6. Decision Node – The system evaluates if a table is free:
   - Available → Proceed to Confirm Booking
   - Unavailable → Show error message to the user
7. Confirm Booking – A reservation is finalized and stored.
8. Show Unavailable Message – Feedback is provided if no table is available.

Model Features:
- Start and end nodes (filled and bullseye circles)
- Correct swimlane structure
- Decision node with labeled flows
- Standard UML action notation

Task 4: Software Architectural Design

The CloudTables-Customer subsystem is designed using a microservices architecture style. Each major functionality is encapsulated in a self-contained service, communicating via RESTful APIs and operating independently. This architecture ensures modularity, scalability, and service isolation. The UML component diagram below models the subsystem structure, showing the service components and their interfaces.



Interfaces & Responsibilities

The following table describes each microservice, its exposed (provided) interface, required interface dependencies, and responsibilities:

Component	Provided Interface	Required Interfaces	Description
API Gateway	IClientEndpoint	All internal services	Routes customer requests to appropriate services via REST APIs.
Restaurant Search	ISearchService	None	Enables customers to search/filter restaurants.
Booking Service	IBookingService	ITableAvailabilityService	Validates and stores table reservations.

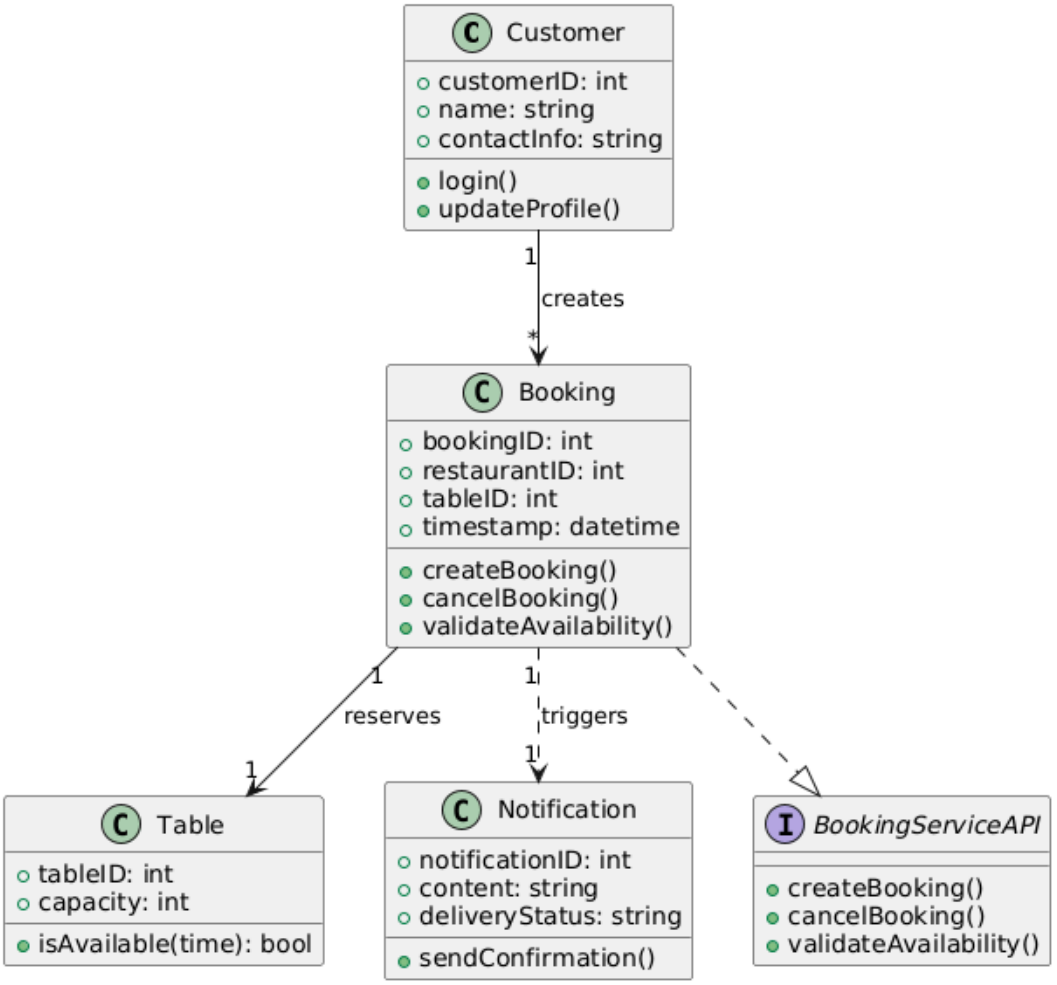
Pre-order Service	IPreOrderService	IMenuService	Manages pre-selection of food for booked tables.
Payment Service	IPaymentGateway	IPaymentProviderService	Processes customer transactions securely.
Review Service	IReviewService	None	Captures and stores user reviews and ratings.
Notification Service	INotificationSender	IBookingService	Sends booking confirmation and status updates via email/sms.

Textual Specification Summary

Each component in the subsystem is modeled as an independent microservice exposing a single public-facing interface. Connectors between components are modeled as RESTful HTTP API calls. The 'API Gateway' provides the external access point for customers using the mobile interface. The 'Booking Service' is central to the subsystem's workflow, coordinating with 'Search', 'Pre-order', 'Payment', 'Notification', and 'Review' services as needed.

Task 5: Software Detailed Design
Task 5a – Structural Model

The structural design of the Booking Service component is presented using a UML class diagram. This model captures the internal classes, attributes, and operations relevant to table booking, and defines the interfaces required and provided. This class diagram ensures that the detailed design supports modularity and aligns with the subsystem's architecture.

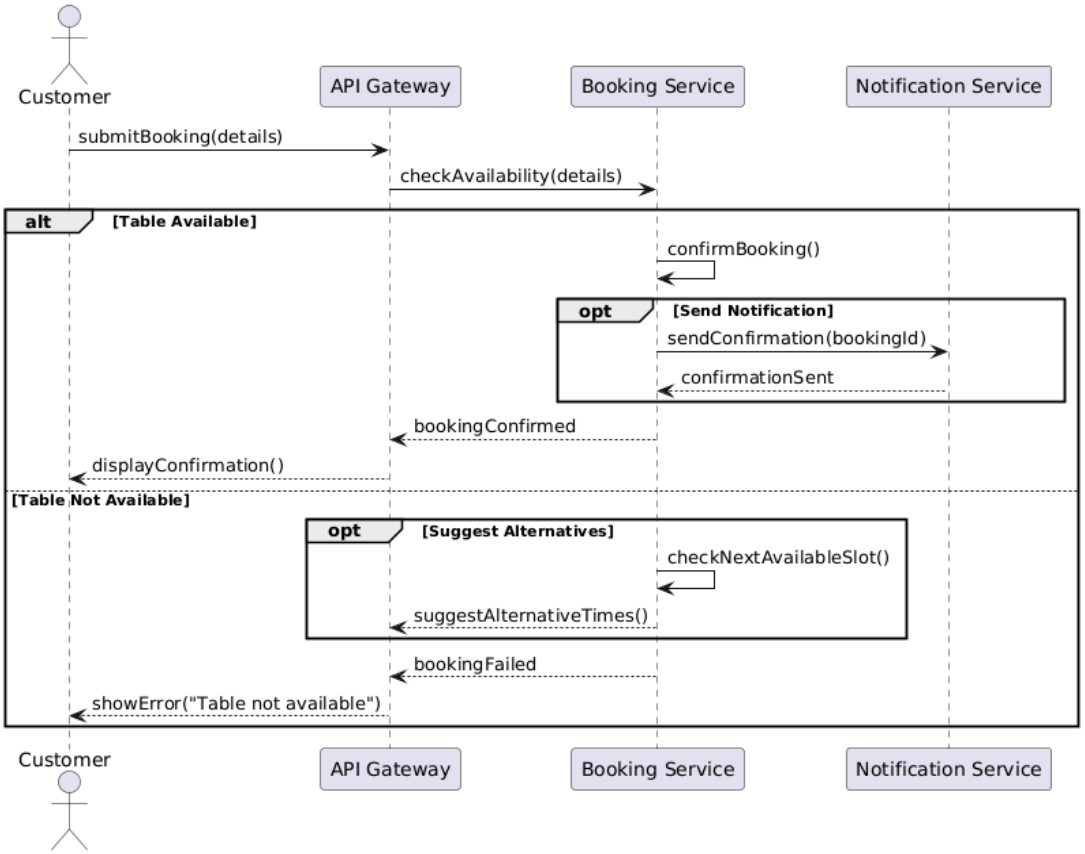


The main classes are:

-Customer: Represents the user making the reservation.
-Booking: Stores all reservation information.
-Table: Holds availability and capacity details.
-NotificationService: Used to send booking confirmation.
-IBookingService Interface: Describes the public API exposed by the Booking component.

Task 5b – Behaviour Model

The behaviour of the Booking Service component is described using a UML sequence diagram. This diagram demonstrates the internal logic when a customer attempts to book a table. It includes conditional logic for table availability and uses interaction fragments to model different outcomes.



Flow Summary:

-The customer submits booking details to the Booking Service.
-The system checks table availability.
- If available, a booking is created and the NotificationService sends confirmation.
- If not available, an error message is returned.